Singly Linked List – Insert After An Element – Space Complexity and Algorithm.

Program:

```
void insertAfterElement(Node **head, int data, int after)
{
    Node *ptr, *loc;
    ptr = (Node *)malloc(sizeof(Node));
    loc = searchInUnsortedList(head, after);
    if (loc == nullptr)
    {
        cout << "Error: Element not found." << endl;
        return;
    }
    ptr->data = data;
    ptr->next = loc->next;
    loc->next = ptr;
}

…..

case 5:
        {
            int data, after;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the element after which to insert: ";
            cin >> after;
            insertAfterElement(&head, data, after);
            break;
        }
```

$Space\ Complexity$

$\underline{Input\ Space}$:

$\underline{A. Parameter - Only\ Definition}$:

$Parameters: The\ function\ takes\ three\ parameters: Node\ **head, int\ data, and\ int\ after.$

$These\ parameters\ require\ a\ constant\ amount\ of\ space, i.e., O(1), because\ they\ are$

$fixed - size\ data\ types\ (a\ pointer\ and\ two\ integers).$

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer), `int data` (an integer), and `int after` (an integer), all of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

• Node **head – one pointer (machine word)

• int data – one integer (machine word)

• int after – one integer (machine word)

Total size is constant, therefore Θ(1) = O(1).

$\underline{B. Full - Data\ (Inclusive) Definition}$:

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same three scalar parameters as above (Θ(1)), plus

• Every node reachable from *head when the function is invoked.

If the list contains n nodes, those nodes occupy Θ(n) bytes.

⇒ Θ(n) = O(n).

$\underline{\textit{Auxiliary Space}}$:

$\boldsymbol{Memory\ Allocation: The\ function\ allocates\ memory\ for\ a\ new\ node\ using\ malloc,}$

$\boldsymbol{which\ is\ a\ constant\ space\ operation. The\ space\ required\ for\ this\ new\ node\ is\ also}$

$\boldsymbol{O(1)\ because\ it\ does\ not\ depend\ on\ the\ size\ of\ the\ input\ linked\ list.}$

$\boldsymbol{Temporary\ Variables: The\ function\ uses\ two\ temporary\ pointers, ptr\ and\ loc.}$

$\boldsymbol{Each\ of\ these\ pointers\ requires\ a\ constant\ amount\ of\ space, O(1).}$

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node *ptr`** **and** **`Node *loc`**: The function declares two local pointers. Each pointer occupies a constant amount of memory.
2. **`malloc(sizeof(Node))`**: Memory for exactly **one** new node is allocated. The size of a single `Node` is constant.
3. **`searchInUnsortedList(...)`**: This helper function is called. As analyzed previously, this iterative search has its own auxiliary space complexity of Θ(1) (it uses one extra `current` pointer).

The total extra memory is the sum of the space for two pointers, one allocated node, and the constant space used by the helper function. Since the sum of several constants is still a constant, the overall auxiliary space does not grow with the size of the list.

Therefore, the auxiliary space complexity is **Θ(1)**.

• One heap allocation for the new node (ptr) → Θ(1).

• Two local pointers (ptr, loc) and a few compiler temporaries → Θ(1).

• The helper call searchInUnsortedList uses only constant auxiliary space (see earlier analysis).

• No recursion. ⇒ Total extra space consumed by the algorithm itself is Θ(1) = O(1).

Algorithm

$InsertAfterElem(START, DATA, AFTERPTR)$

$\left((Node *)malloc(sizeof(Node))\ is\ availability\ of\ space\ for\ insertion\ of\ Data,\ hence\ ,\ lets\ name\ it\ AVAIL.\right)$

1. $[ALLOCATE\ NEW\ NODE]$ $\quad Set\ PTR := AVAIL$
2. $Set\ LOCPTR := SrchInUnsrtdList(START, AFTERPTR)$

$- -$

$Translating\ \ SrchInUnsrtdList(START, AFTERPTR)$

$Set\ AFTERPTR := KEY$

$SrchInUnsrtdList(START, KEY)$

$Set\ CURRPTR := START\ [Sets\ CURRPTR\ pointer\ to\ \ NODE\ pointed\ by\ HEAD\ or\ START]$

$Repeat\ while\ CURRPTR\ \neq NULL\ AND\ INFO[CURRPTR\ ]\ \neq KEY, then:$

$Set\ CURRPTR\ := LINK[CURRPTR]\ [It\ sets\ CURRPTR\ pointer\ to\ the\ address\ of\ NODE\ where\ KEY\ is\ located]$

$[Otherwise\ , if\ KEY\ is\ not\ found\ then, CURRPTR = NULL, LOOP\ ends]$

$[End\ of\ Loop]$

$return\ CURRPTR$

$- - - - - - - - - - - - - - - -$

$Translating\ Return$

$Set\ LOCPTR := \ CURRPTR$

$- -$

$- -$

3. $[CURRPTR\ is\ NULLPTR?\ ]if\ LOCPTR = NULL, then:$

$Write:``\ Error: Element\ not\ found.``$

$Return\ and\ exit.$

$[END\ OF\ IF\ STRUCTURE]$

$[Set\ DATA\ to\ INFO\ and\ LINK\ pointed\ by\ PTR\ and\ put\ that\ NODE\ to\ desired\ location]$

4. $Set\ INFO[PTR] := DATA\ [The\ NODE\ pointed\ by\ PTR\ , INFO\ part\ of\ NODE\ set\ to\ DATA]$
5. $Set\ LINK[PTR] := LINK[LOCPTR]\ [LINK\ of\ NODE\ pointed\ by\ PTR\ assigned\ to\ LINK\ of\ NODE\ pointed\ by\ LOCPTR]$
6. $Set\ LINK[LOCPTR] := PTR\ [Address\ stored\ in\ PTR\ as\ Value\ set\ to\ LINK\ of\ NODE\ pointed\ by\ LOCPTR\ ]$

$\left[Note\ , LINK\ of\ NODE\ pointed\ by\ LOCPTR\ will\ now\ point\ to\ that\ address\ stored\ in\ PTR,\ This\ will\ put\ that\ NODE\ pointed\ by\ PTR\ in\ the\ desired\ location.\right]$